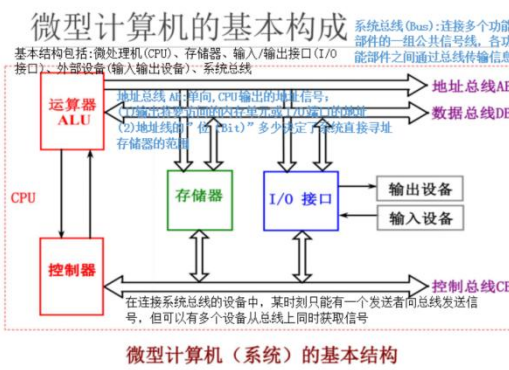
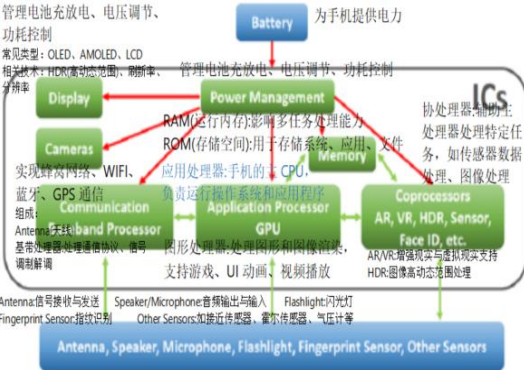




控制总线CB:

双向, CPU对存储器、I/O接口进行控制和联络。

- 输出控制信号: CPU发给存储器或I/O接口的控制信号。如, 微处理器的读信号RD, 写信号WR等。
- 输入控制信号: CPU通过接口接受的外设发来的信号。如, 外部中断请求信号INTR、非屏蔽中断请求输入信号NMI等。
- 控制信号间相互独立, 表示方法: 采用能表明含义的缩写英文字母符号。按照惯例, 若符号上有一横线, 则表示该信号为低电平有效, 否则为高电平有效。

数据总线DB:

双向, 数据在CPU与存储器(或I/O接口)间传送;

- CPU读操作时, 外部数据
- CPU写操作时, CPU数据
- 数据线的多少决定了一次能够传送数据的位数(8, 16, 32, 64);
- 串行-并行
- CPU通过AB(地址总线)上不同的地址确定读写的存储(接口)单元, 经由DB(数据总线)与存储器(或I/O接口)进行数据传输。

关于微机需要区别的概念

微型计算机(系统)的基本结构

微处理器

简称MP(Micro Processor), 也称 μP 。

是微机的核心部件。通常称为中央处理单元CPU (Central Processing Unit), 包括:

- 运算器ALU (Arithmetic Logic Unit)
- 控制器CU (Control Unit)
- 寄存器阵列R (Registers)
- 内部总线等电路

——集成在一片硅片上

微型计算机的基本构成

1Mbit并行EEPROM

2k bit串行EEPROM

3、存储器

分为程序存储器和数据存储器两类。

片上寄存器 (RAM、ROM), 内存, 硬盘

根据速度-容量之间的折中, 分成多个层次

4、I/O接口

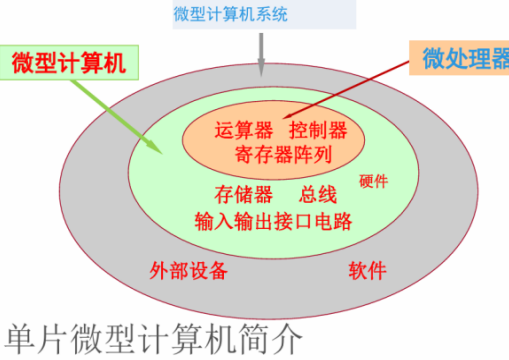
主要用于CPU和外部设备之间交换数据。

并行口

串行口

USB口等

- (1) 微处理器即CPU —— 计算机、电子系统的核心部件
- 将运算器、控制器集成在一片芯片上。功能: 指令译码, 读写数据; 响应中断请求; 控制。X86、ARM、PowerPC、MIPS
- (2) 微型计算机
- 在CPU基础上配置存储器、I/O接口电路、系统总线。
- (3) 微型计算机系统
- 以微机为主体, 配置系统软件和外设。软件部分包括系统软件(如操作系统)和应用软件(如字处理软件)。



3. 典型的微型处理器

单片微型计算机(单片机)就是将计算机的核心部分:

- 中央处理器CPU
- 存储器
- 通用I/O接口(电路)
- 典型外设
- 内部总线

——集成在一块芯片上的计算机

单片微型计算机简介

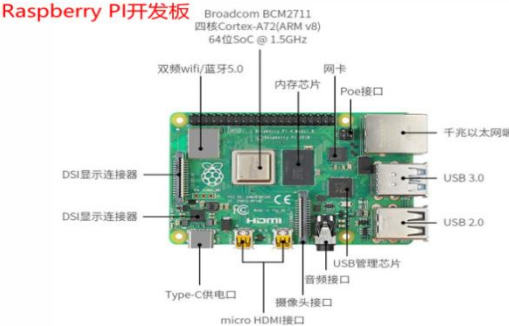
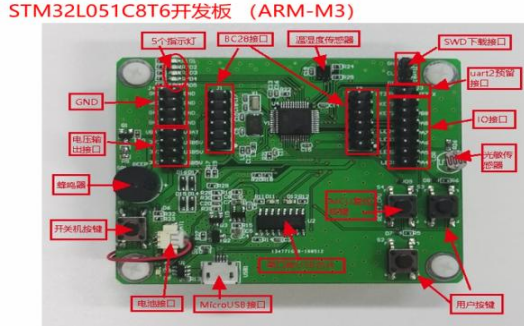
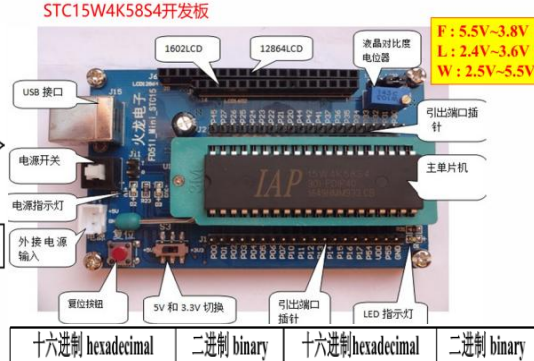
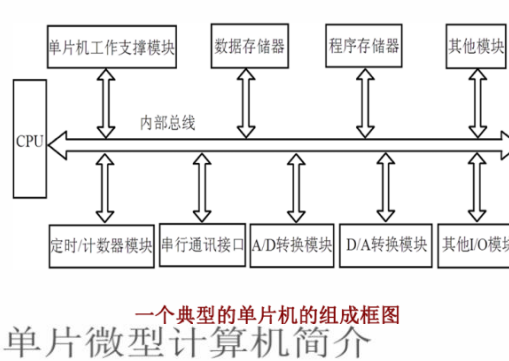
单片微型计算机简介

1、单片机的基本定义

在一块芯片上集成了中央处理单元(CPU)、存储器(RAM/ROM等)、定时/计数器以及多种输入/输出(I/O)接口的比较完整的数据处理系统。

2、单片机名称的来源

- ◆ 早期的英文名称是Single-chip Microcomputer, 即单片微型计算机, 简称单片机。
- ◆ 后来称之为微控制器 (Microcontroller), 这也是目前比较正规的名词。
- ◆ 我国学者或技术人员一般使用“单片机”一词。



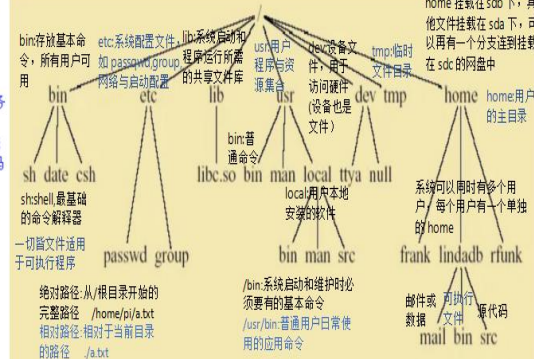
十六进制 hexadecimal	二进制 binary	十六进制 hexadecimal	二进制 binary
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

7-bit 值	用途	记忆口诀
0000 000	通用广播	0x00
0000 001 - 0000 111	CBUS 等扩展	0x01-0x07
1111 000 - 1111 111	10-bit 扩展前缀	0x78-0x7F

Unix Philosophy Linux Philosophy Linux 哲学

多用户/多任务: 多个用户可以同时登录并运行多个程序而互不干扰

- * **Multiuser / Multitasking**
- * 工具箱方法: 每个程序只做好一件事, 用户可通过 pipe 将它们组合完成复杂任务
- * **Toolbox approach (make each do one thing well)**
- * 灵活性/自由: 用户可以自由选择工具、脚本语言、shell 等, 甚至可以修改系统
- * **Flexibility / Freedom**
- * 简洁性: 程序应该尽量保持精简, 减少错误且提高可维护性
- * **Conciseness (small is beautiful)**
- * **Everything is a file** 接口访问, 方便系统设计和用户操作
- * 文件系统有路径、结构, 进程有诞生、运行、终止的过程, 文件和进程为核心
- * **File system has places, process has life**
- * 由程序员设计、为程序员服务的系统, 优先考虑效率、可脚本化、可组合性
- * **Designed by programmers for programmers**
- * Even of the programmers



总共 16 个地址 = 1 + 7 + 8 = 16

Unix Programs

“命令”抽取成同一套 exec 接口。Shell 只负责解析命令行，决定走哪条通路，然后统统扔进内核——所以 Shell 既“特殊”（它是用户与内核的默认门户）又“普通”（它本身不过是个进程）。

- 解释器角色:读入→解析→执行 (通过系统调用与内核打交道)
- * **Shell is the command line interpreter**
- 强调通用性:4)可存放在/bin/sh, /bin/bash, /usr/bin/zsh 等, 是一个可执行文件
- * **Shell is just another program**
(2)可以被 execve 像启动 ls 一样启动, 也可以被用户随时更换(chsh)
(3)内核以子进程方式启动普通进程的同一套机制
- * **A program or command interacts with the kernel may be any of:** 区别在谁负责生成系统调用字节码
shell 自己, Shell 进程直接调用对应 C 函数, 然后陷入内核; 不产生新进程
* **built-in shell command** cd, pwd, echo (部分实现), umask
- 外部解释器(python3, sh) shell fork → execve("python3"), 1-1 Python 字节码 → Python 运行时 → 系统调用
CPU Shell fork → execve("/bin/ls", ...) → 内核加载 ELF → CPU 执行机器码 → 系统调用
- * **compiled object code file** /bin/ls, /usr/bin/gcc

Set User Environment (1)

- stty** 作用:查询或设置当前终端的行规程参数, 比如按哪个键删除
- * **stty - terminal control** 字符, 按哪个键删除行
终端是内核里的 tty 行规程, stty 是用户空间修改这些规程的唯一命令
reports or sets terminal control options
configures aspects of I/O control
- * **syntax: stty attribute value**
example: stty erase ^H; stty kill ^U;
变量分 shell 变量与环境变量; 只有 export 后才会被 fork 出
- * **Environment variables** 的子进程继承
login 程序从 /etc/passwd 第 6 字段读取后与 shell 环境。
- * **HOME** 影响: cd 不带参数就进 \$HOME; 很多程序 (vim, bash, git) 默认在这里找配置 dot-file, 手动改会找不到原配置
- * **PATH** 号分: 分隔的目录列表, 从左到右搜索可执行文件。
推荐添加法: PATH=\$HOME/bin:\$PATH 保证个人脚本优先
- * **SHELL** 又设置系统路径。
login 程序根据 /etc/passwd 最后一字段填写。
- * **env - show current Env values**
排查: 登录后 env | grep -E "\$PATH\$HOME" 看变量是否被覆盖。
清理: env -i bash -l 启动一个空环境的全新 bash, 方便调试。

Identifying Files in the Tree

- 绝对路径:从根目录/开始, 逐级写出文件的位置
- * **Identifying files using full path names**
永远以/开头; 不依赖你当前所在目录(位置不变); 系统唯一指向该文件的位置
- * **Full path names always begin with /**
\$HOME: 你的 home 目录, 例如 /home/student; 等价于 \$HOME
- * **\$HOME, ~, and EVs**
环境变量 (environment variables) 也可扩展为路径
Example: /usr/dt/bin/dtterm
- 相对路径:基于你当前所在目录来描述文件路径
- * **Identifying files using relative path names**
不以/开头; 依赖当前所在的工作目录(pwd); 写起来简单, 但可能在不同目录下失效
- * **Dependent on your location in the file system**
.: 当前目录; ..: 上一级目录 (parent directory)
- * **and ..**
绝对路径=GPS 坐标, 全世界唯一
相对路径=指路说明, 必须知道你在哪里
- * **Example: .././include/define.h**

Directory Commands

- pwd:显示你当前所在的目录路径**
- * **pwd** [jiangjunmin@eex001 ~]\$ pwd
/home/jiangjunmin
- * **mkdir directory-list**
mkdir: 创建一个或多个目录
Example: mkdir personal manage
- * **cd [directory]**
cd: 进入(切换到)指定目录
rmkdir: 删除一个或多个目录(里面不能有文件或子目录)
Example: rmkdir personal manage
- * **cd Full Path Names**
等价, 回到用户的主目录
cd = cd ~ = cd \$HOME
- * **cd Relative Path Names**
进入其他用户的主目录(有权限) 波浪号可以 cd thompson 指定用户

Permissions

- * **Access permissions (characters 2-10):**
first 3: user/owner
second 3: assigned Unix group
last 3: others
- * **Permissions are designated:**
Size or device number
- * **r** read permission
Modified time
- * **w** write permission
File name
- * **x** execute permission
- * **-** no permission

Change Permissions on File

- * **chmod [options] file**
using + and - with a single letter: u user owning file
g those in assigned group
o others
- * **Examples**
chmod u+w file gives the user (owner) write permission
chmod g+rw file gives the group read/write permission
chmod go-x file removes execute permission for group

Command Line Structure (2)

- 标准不统一, 同一套 options 在不同命令里可能风格迥异
- * **Not all Unix commands will follow the same standards**
command: 可执行文件的名字 (内置、脚本、二进制均可)。
option: 以 - 或 -- 开头, 告诉命令“怎么做”。
argument: 告诉命令“对谁做”。
- * **Options and syntax for a command are listed in the 'man page' for the command**
- * **Be aware that Unix is case sensitive**
“谁干, 怎么干, 对谁干”——对应 command, options, arguments.
例: ls -l /home
谁干: ls; 怎么干: -l (长格式); 对谁干: /home
- * **Use lower case**
Unix 区分大小写, 命令名本身必须小写
- * **To terminate a command**
^C interrupt 向当前前台进程发 SIGINT, 强制中断
^D can log a user off; frequently disabled 向终端发 EOF, 会结束当前 shell; 若系统没禁用, 则直接 logout

Set User Environment (2)

- * **setenv (csh)**
在 csh 中创建或修改环境变量并立即导出给该 shell 启动的子进程 setenv 变量名 变量值
setenv PATH ~/bin:/bin:/usr/bin 不使用等号
PATH:告诉系统到哪些目录去查找可执行命令。当前目录, 使用命令系统会按照这些路径顺序查找
- * **setenv MANPATH /usr/share/man:/usr/local/man**
MANPATH 指定 man 命令搜索手册页的位置 /usr/share/man 与 /usr/local/man 是常见的 man 文件目录
- * **setenv MANPATH \$HOME:/man:\$MANPATH**
\$HOME/man 为用户自定义的 man 目录 \$MANPATH 表示在原有 man 搜索路径之后添加
- * **set and export (sh, bash)**
bash 不使用 setenv, 先给变量赋值, 再使用 export 将变量导出为“环境变量”
PATH=/usr/local/bin:/home/jack/bin; export PATH
export PATH=/usr/local/bin:/home/jack/bin
export PATH=\$PATH:/opt/acroread/bin
\$PATH 表示保留原有路径 /opt/acroread/bin 表示在后面追加 Acrobat Reader 的可执行路径
- * **alias**
给复杂或常用的命令创建一个短名字 完整命令要用 ' ' 包起来, 别名不能与系统命令同名 (否则覆盖)
- * **syntax: alias aname pname**
- * **alias dir 'ls -al | more'**
dir 这个新命令, 执行时等于 dir 子命令: 列出所有文件(包括隐藏文件), 用 more 分页显示
- * **alias ls 'ls -al *.c | more'**
ls 查找文件路径: 文件名 -inode number
-inode -数据块

File Structure on Disk

- 磁盘被挂载到文件树的某个位置 Linux 只有一个统一的文件树
- 不同的磁盘、分区不会像 Windows 那样出现 C:, D:
- * **Disks mounted to positions in file tree**
磁盘最小的读写单位不是文件, 而是块, 常见块大小: 512bytes (1KB)
- * **A disk is divided into blocks**
512, 1024, 2048 bytes are common block sizes
- * **A disk partition divided into three major sections**
超级重要的小块区域, 包含: 文件系统类型 (ext4, nfs), block 大小, 总块数, inode 数量; 被使用/空闲的 superblock 块数量; 文件系统的状态 (是否正常运行) 如果坏掉, 文件系统无法识别每个文件都对应一个 inode (索引节点), inode 里面存放: 文件的元数据: 拥有者: 权限: 创建时间: 修改时间: inodes 间: 文件大小: 指向数据块的地址 (文件数据在哪里) 不存在文件名
真正存储文件内容的地方, 比如: 文本内容, 图片数据, 可执行程序的二进制, 文件最终以块的形式存储在 data blocks

More left for later on System Administration ls - List Contents of Directory

- * **Syntax: ls: 列出目录中的文件与子目录**
- * **ls [-aAbcCdffGgilMnoprRstuxl] [file ...]**
- * **Frequently used options:**
a: 显示所有文件, 包括以 . 开头的隐藏文件 默认情况下, 以 . 开头的文件会被隐藏。
-a all, including files begin with '.' 不显示
d: 显示目录本身, 而不是目录内容 若不加, 会列出目录里的具体内容, 加了才会把目录
l: 以长格式显示 输出中会包含: 权限 (rwx), 硬链接数, 文件所有者, 所有者组, 文件大小, 修改时间: 文件名称
-l long 间, 文件名 用途: 查看文件权限、大小、时间等详细信息
F: 用特殊符号标记文件类型 / 目录: 可执行文件 @ 符号链接 * 管道
R: 递归列出目录 (包含子目录) 用途: 快速递归目录、文件、可执行程序
-R recursive 会显示当前目录 - 子目录 - 子目录的目录等所有内容
t: 按修改时间排序 (最新的排前面) 用途: 查看整个目录树结构
-t time 常和 -l 搭配, 用于查找最近修改的文件

Types Displayed under Linux

- * **alias ls**
- ls --color=auto**
blue directories
white/black text files
cyan links
green executables
red compressed archives
pink images
yellow devices
(flashing) red broken links
- * **ls -F**
nothing regular file directory
* executable file @ link
= socket | named pipe

Permission in Numeric

- * **chmod [options] file**
using numeric representations for permissions:
r = 4
w = 2
x = 1
Total: 7
chmod 7 7 7 filename
user group others
Gives user, group and others the r, w, x permissions by this representation

Login

- * **Your user name**
- * **Your password**
- * **Control keys (^) perform special functions**
删除左边一个字符 输错用户名/密码时直接回退, 不用回车
H or Backspace erase a character
Ctrl + C 密码打出太长发现全错, 一键清空当前行
Ctrl + Z cancel line
暂停屏幕输出 cat 大文件滚屏太快, 先“冻结”看某一段
暂停屏幕输出 再按继续键, 俗称“软件流量控制”
Ctrl + Q restart display
中断当前前台任务 再输入用户名/密码时直接回退, 不用回车
发送 EOF (文件结束符) 在 login 提示下按 ^D 立即断开连接 (等于 logout exit); 某些系统会再确认
^D signal end of file
原样转移下一个控制键 需要把 ^M 当普通字符写进密码或文件时, 先按 ^V 再按 ^M
^V treat following control character as normal character

Commands for Useful Info

- who: 显示当前系统上所有已登录用户的信息, 包括用户来源、终端类型等**
- who am i: 显示当前登录会话本身的登录信息, 而不是当前的有效用户身份**
- whom am i: 显示当前有效用户身份**
who, who am i, whoami
- ps 显示当前用户的正在运行的程序 (默认只显示与当前终端相关的进程)**
piano% who
opc console Apr 16 08:02 (:0)
liangzhs pts/2 Apr 16 15:49 (10.14.111.198)
yu jun dtremote Apr 18 19:37 (10.14.111.244:0)
wangjs pts/10 Apr 18 12:47 (10.14.111.132)
- ps 显示当前用户的正在运行的程序 (默认只显示与当前终端相关的进程)**
piano% ps
PID TTY TIME CMD
4059 pts/38 0:00 csh
4061 pts/38 0:01 ps
4286 pts/38 0:00 xterm

File & Directory Naming Guidelines

- 这些字符在 shell 中有特殊功能, 如果用在文件名里, 会导致: 命令解析错误; 无法正常访问文件; 必须用引号或转义字符才能操作, 极不方便
- * **Don't Use Meta Characters**
* \ " ' * ; ? { } () [] ~ ! \$ < > | & #
* <http://en.wikipedia.org/wiki/Punctuation>
/: 目录分隔符 *: 通配符 wildcard *: 单字符通配符
\$: 表示用户 home 目录 \$: 变量引用 *: 命令分隔
#: 后台执行 > / <: 重定向 *: 引用
#: 注释 0 0 0: 模式匹配、数组、命令分组
- * **Do Use**
* a-z A-Z
* 0-9
* . _
* Remember Unix is Case Sensitive!
- List Directory Contents**
第一个字段 (10 个字符) 是: [文件类型][所有者权限][组权限]
* Try to run 'ls -l' [其他用户权限]
- Type field (first character)**
该项是一个目录 (文件表), 即: 可以包含文件、子目录
d the entry is a directory;
l the entry is a symbolic link; 指向另一个文件或目录。
b the entry is a block special file;
c the entry is a character special file;
- the entry is an ordinary file;

File Maintenance Commands

- 修改文件或目录的访问权限 (read/write/execute)**
- * **chmod change the file or directory access permissions (mode)**
- * **umask get or set the file mode creation mask**
查看或设置新建文件或目录默认权限的掩码
mask 新文件权限=默认权限-umask
改变文件或目录的所属用户组 需要有写权限或 root 权限
* **chgrp change the group of the file**
改变文件或目录的拥有者 普通修改自己, root 修改任意
* **chown change the owner of a file**
- * **chmod 750 filename**
gives the user read, write, execute
gives group members read, execute
gives others no permissions
- * **chmod 640 filename**
gives the user read, write
gives group members read
gives others no permissions

Display Commands

将文本字符串输出到标准输出(stdout)
在终端中,stdout 是屏幕,echo 输出文本到屏幕

- * **echo** echo the text string to stdout
- 连接并显示文件内容,会按顺序把多个文件接在一起输出
- * **cat** concatenate (list)
- 显示文件前面 10 行,默认 10 行,可以用-n 指定行数 head -n 20 file
- * **head** display first 10 (or #) lines of file
- 显示文件最后 10 行,默认 10 行,可以用-n 指定行数 tail -n 20 file
- * **tail** display last 10 (or #) lines of file
- 分页浏览文本文件(从头往后)
- * **more** browse or page through a text file
- SPACE:向下翻一整页 RETURN:向下滚一行 b:返回上一页
- * **SPACE RETURN b q** q:退出 more

find - Find Files

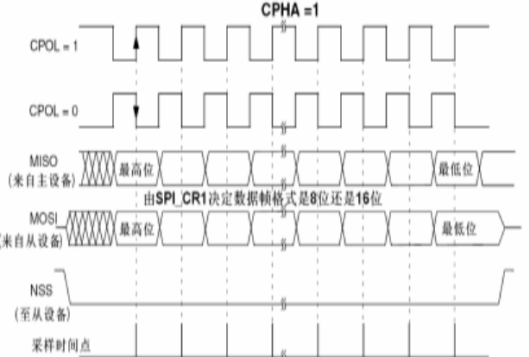
directory-list:从哪些目录开始搜索(可以多个); tests(条件):按名称、时间、类型、大小等筛选文件; actions(动作):对找到的文件执行打印、删除、执行命令等

- * **find directory-list [options] [actions] [...]** -exec rm:非常危险会自动删除;-ok rm 比较安全,会逐个询问
- * **find . -name Net*.c -print**
- * **find . -newer empty -print**
- * **find /usr/local -type d -print**
- * **find . \ (user Bill -o -size +10000c \) -print -exec rm { } \;**
- * **find ~Bill ~Denis -size +1000 -atime 30 -ok rm { } \;**

Utility - diff

对两个文本文件进行逐行比较,输出的是哪些行不同、哪些行新增、哪些行删除

- * **diff file1 file2** 有(被新增) (空白):两个版本都保留
- * **Line-by-line differences between pairs of text files**
- * **Often used for comparing text file (source code, netlist, data list, etc.) in different versions.**
- * **tkdiff vs. windiff**



SPI 数据发送接收

SP 主机和从机都有一个串行移位寄存器,主机通过向它的 SPI 串行寄存器写入一个字节来发起一次传输。

1. 首先拉低对应 SS 信号线,表示与该设备进行通信。
2. 主机通过发送 SCLK 时钟信号,来告诉从机写数据或者读数据。
这里要注意, SCLK 时钟信号可能是低电平有效,也可能是高电平有效,因为 SPI 有四种模式,这个我们在下面会介绍。
3. 主机(Master)将要发送的数据写到发送数据缓存区(Memory),缓存区经过移位寄存器(0~7),串行移位寄存器通过 MOSI 信号线将字节一位一位地移出去传送给从机,同时 MISO 接口接收到的数据经过移位寄存器一位一位地移到接收缓存区。
4. 从机(Slave)也将自己的串行移位寄存器(0~7)中的内容通过 MISO 信号线返回给主机,同时通过 MOSI 信号线接收主机发送的数据,这样,两个移位寄存器中的内容就被交换。

- 1. **原码 & 反码 (范围完全相同)**
 - 正数最大值: 符号位 0 + 数值位全 1 (15 个 1) → 二进制 0111111111111111
 - 对十进制: $2^{15} - 1 = 32767$ (因为 15 位数值位最大是 $2^{15} - 1$)。
 - 负数最小值: 符号位 1 + 数值位全 1 (15 个 1) → 二进制 1111111111111111
 - 对十进制: $-(2^{15} - 1) = -32767$ (数值位全 1 是最大值,加负号后是最小负数)。
- 特殊编码: +0 (0000000000000000) 和 -0 (1000000000000000),两者数值上都显 0,但编码不同(浪费 1 个编码)。
- 继续范围: [-32767 ~ 32767] (共 $2^{16} - 1 = 65535$ 个不同编码)。
- 2. **补码 (范围更合理,无 -0)**
补码的核心改进是:将原码的“-0”编码(1000000000000000)重新定义为「最小负数」,消除了编码浪费。
 - 正数最大值: 和原码一致 → 0111111111111111 → 十进制 32767。
 - 负数最小值: 符号位 1 + 数值位全 0 (15 个 0) → 二进制 1000000000000000
 - 对十进制: $-2^{15} = -32768$ (这个编码无法用「原码/反码规则」表示,是补码独有的)。
 - 无 -0: 补码中 0000000000000000 是唯一 0 编码,1000000000000000 显 -32768。
 - 继续范围: [-32768 ~ 32767] (共 $2^{16} = 65536$ 个不同编码,覆盖所有 16 位二进制组合)。

File Manipulation Commands

-f:force,覆盖目标文件时不询问(强制复制) -i:interactive,覆盖目标文件前询问确认 -p:preserve,保留源文件的属性(时间戳,权限,owner) -R/-r:recursive,递归复制目录

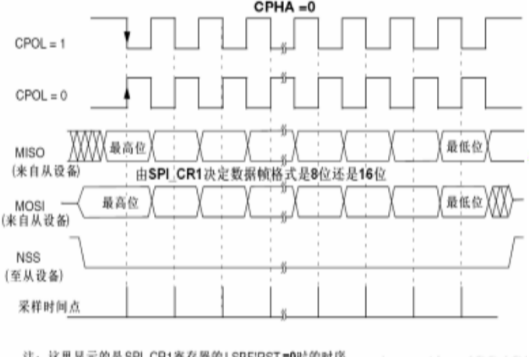
- * **cp** copy file
- * **cp [-fRpRr] source_file target_file**
- * **mv** move (or rename) file
- * **mv [-fi] source target_file**
- * **mv [-fi] source ... target_dir**
- * **rm** remove (delete) a file (directory)
- * **rm [-f] [-i] file ...**
- * **rm -rR [-f] [-i] dirname ... [file ...]**
- * **Meaning of each option here**

Compression/decompression

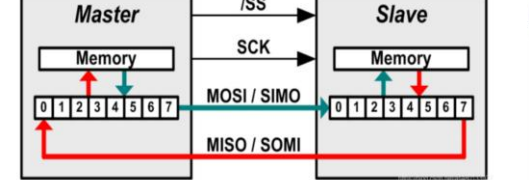
compress 是早期 Unix 的 LZW 压缩,现代系统普遍使用 gzip 替代
-c:将压缩/解压结果输出到 stdout(不生成文件) -f:强制覆盖已有文件
* **compress -c file1 [-b bits] [file ...]**
* **uncompress [-cfv] [file ...]** 解压(同 gunzip)
* **.Z** as suffix
-c:输出到 stdout,不删除原文件 -d:解压(同 gunzip)
-f:强制覆盖现有文件 -r:递归压缩目录下所有文件
压缩等级:1-9,1最快压缩,压缩率最低,9:最慢压缩,压缩率最高
* **gzip [-cdmLnRtvV19] [-S suffix] [file ...]**
文件名称、软链接 -n:不保存原文件名与时间戳 -N:保存原文件名与时间戳(默认) -S suffix:指定自定义扩展名
信息显示 -v:verbose,显示压缩比率等信息 -V:显示版本号 -h:显示帮助
* **gzip filename filename.gz->filename**
* **gzip -d filename filename.gz->filename**
* **gzip -h** help for gzip
* **gunzip**
* **bzip2 / bunzip2**

Utility - grep

- * **grep/egrep/fgrep** - search the argument for all occurrences of the search string (get REP)
 - The **grep** utility is used to search for regular expressions in Unix files.
 - **fgrep** searches for exact strings (not pattern).
 - **egrep** uses "extended" regular expressions.
- * **grep regexp file**
 - **grep static ./*.c**
 - **grep ^# *.c**
 - **man ls | grep -ci permission**



注:这里显示的是 SPI_CR1 寄存器的 LSBFIRST=0 时的时序



SP 只有主模式和从模式之分,没有读和写的说法,外设的写操作和读操作是同步完成的。
如果只进行写操作,主机只需忽略接收到的字节;
反之,若主机要读取从机的一个字节,就必须发送一个空字节来引发从机的传输。
也就是说,你发一个数据必然会收到一个数据;
你要收一个数据也必须先发一个数据。

I2C (Inter-Integrated Circuit) 是一种常用于短距离通信的串行总线协议。由飞利浦公司 (现为 NXP) 于 1980 年代初期开发,广泛应用于微控制器、传感器、显示器等设备之间的通信。I2C 协议是多主机、多从机的同步串行通信协议,具有较低的硬件要求,且支持多个设备共享同一总线。以下是 I2C 协议的详细介绍:

1. 基本结构

- I2C 通信通过两条线进行数据传输:
- **SDA (Serial Data Line)**: 串行数据线,用于传输数据。
 - **SCL (Serial Clock Line)**: 串行时钟线,用于同步数据传输。
- I2C 协议采用主从模式,其中:
- **主设备 (Master)**: 负责发起通信,控制时钟 (SCL 线),并决定何时开始与哪个从设备进行数据交换。
 - **从设备 (Slave)**: 响应主设备的命令,按照主设备的要求传输数据。

In and unlink

ln:创建链接(默认硬,加-s 表示符给文件建立一个链接(link)号); Hard link:同一个文件的另一个名字;Symbolic link:快捷方式,指向路径

- * **ln [options] source target** - Link to another file
Linux 有两种链接:1.Hard link(硬链接) 2.symbolic link(软/符号链接)
特点:类似 symbolic link Windows 的快捷方式;保存的是路径名;删除源文件变成死链;可链接目录;可跨文件系统
特点:文件内容的另一条指针;指向同一个 inode(真正的文件数据结构);删除其中一个不影响其他;不可链接目录(为避免死 * hard link 循环);不能跨文件系统(inode 不同) 移除文件的一个链接(文件名),只能删除一个且不能为目录
- * **unlink filename** - Remove the link

Utility - wc

wc(word count):统计文件中的行数(l)、单词数(w)、字符数(c)

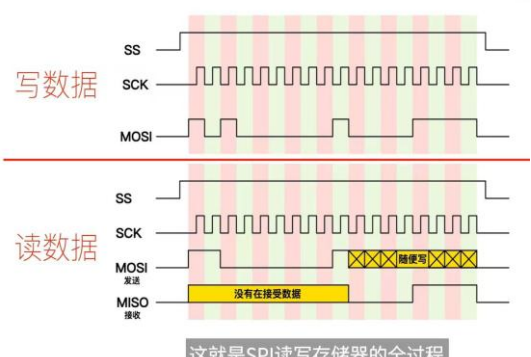
- * **wc** - display a count of lines, words and characters in a file
- * **-C**
- * **-l**
- * **-W** wc-l 只统计行数
- * **ls | wc -l**

Utilities

判断文件类型

- * **file** - determine file type
- * **sort** - sort, merge, or sequence check text files
- * **which** - locate a command; display its pathname or alias
- * **whatis** - search the whatis database for word
- * **grep** - search the whatis database for string
- * **spell** - report spelling errors of file
- * **look** - find words in dictionary

Other standard utilities in /usr/bin, /usr/sbin



这就是 SPI 读写存储器的全过程

连接电容是什么

I²C 上拉电阻阻值计算 (最大值)

- 总线电容 C_b
 - 总线电容 (C_b) = 线缆电容 (C_w) + 连接电容 (C_c) + 引脚电容 (C_i)
 - 上拉电阻的最大值,和总线电容决定了信号上升和下降的时间。
- $V_{IL} = 0.3V_{DD}$, $V_{IH} = 0.7V_{DD}$
 - $0.3 \times V_{DD} = V_{DD} (1 - e^{-\frac{t_1}{\tau_1}})$, $t_1 = 0.3566749 \times RC_b$
 - $0.7 \times V_{DD} = V_{DD} (1 - e^{-\frac{t_2}{\tau_2}})$, $t_2 = 1.2039729 \times RC_b$
 - $\tau_1 = t_2 - t_1 = 0.8473 \times RC_b$
- $R_{P(max)} = \frac{t_1}{\tau_1}$
 - $\tau_1 = 1000ns$ (标准模式), $300ns$ (快速模式), $120ns$ (快速模式+)
 - C_b 是一个对总线电容的估算值。

最小值 $\frac{V_{DD} - V_{OLmax}}{I_{OL}}$ $\frac{3.3V - 0.4V}{3mA}$

CH340T 管脚定义:

TXD:输出,串行数据输出(CH340R 型号为反相输出)
RXD:输入,串行数据输入,内置可控的上拉和下拉电阻
XI:输入,CH340T/R/G 晶体振荡的输入端,需要外接晶体及电容
XO:输出,CH340T/R/G 晶体振荡的输出端,需要外接晶体及电容
RST#:输入,CH340B 外部复位输入,低电平有效,内置上拉电阻
UD+:USB 信号,直接连接到 USB 总线的 D+ 数据线
UD-:USB 信号,直接连接到 USB 总线的 D- 数据线
CTS#:输入,MODEM 联络输入信号,清除发送,低(高)有效
RTS#:输出,MODEM 联络输出信号,请求发送,低(高)有效
DSR#:输入,MODEM 联络输入信号,数据装置就绪,低(高)有效
DTR#:输出,MODEM 联络输出信号,数据终端就绪,低(高)有效
RI#:输入,MODEM 联络输入信号,振铃指示,低(高)有效
DCD#:输入,MODEM 联络输入信号,载波检测,低(高)有效

R232:输入,CH340T/R/G辅助RS232使能,高有效,内置下拉电阻
NOS#(16管脚芯片无):输入,禁止USB设备挂起,低电平有效,内置上拉电阻
ACT#(16管脚芯片无):输出,USB配置完成状态输出,低电平有效
IR#(16管脚芯片无):输入,CH340R串口模式设定输入,内置上拉电阻,低电平为SIR红外串口,高电平为普通串口
CKO(16管脚芯片无):输出,CH340T时钟输出
VCC:电源,正电源输入端,需要外接0.1uF电源退耦电容
GND:电源,公共接地端,直接连到USB总线的地线
V3:电源,在3.3V电源电压时连接VCC输入外部电源,在5V电源电压时外接容量为0.01uF退耦电容

* man

- * Manual sections
- * man man

NAME

man - find and display reference manual pages **SYNOPSIS**
 man [-] [-adFlrt] [-M path] [-T macro-package] [-s section] name ...
 man [-M path] -k keyword ... *man -T pdf ls > ls.pdf 生成ls命令手册PDF*
 man [-M path] -f file ...

DESCRIPTION

The man command displays information from the reference manuals. It displays complete manual pages that you select

-M 临时覆盖默认路径

* Spacebar - display next page

* b - previous page

* q - quit

* xman - man has X GUI on Solaris

* KDE/Gnome help browsers on Linux

* Web resources

图形界面帮助浏览器

Other Paragraphs in 'man' man man

man 命令手册的末尾部分

USAGE

ENVIRONMENT VARIABLES

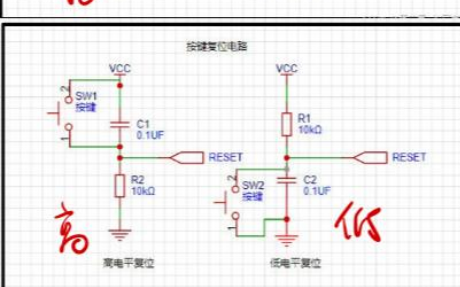
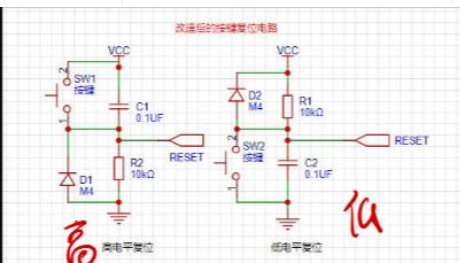
EXIT STATUS

SEE ALSO

apropos(1), cat(1), col(1), eqn(1), more(1), nroff(1), refer(1), tbl(1), troff(1), vgrind(1), whatis(1), catman(1M), attributes(5), environ(5), eqnchar(5), man(5), sgml(5)

BUGS

符号 / 表达式	核心含义	典型使用场景 (配合命令)
.	当前工作目录 (终端 pwd 命令显示的目录)	- ls .: 列出当前目录下所有内容 - cp /home/file.txt .: 将文件复制到当前目录 - cat ./config.conf: 查看当前目录下的配置文件 (./可简化为.)
..	当前目录的上级目录 (父目录)	- cd ..: 切换到上一级目录 - ls ../docs: 查看上级目录中 docs 文件夹的内容 - mv test.txt ../backup/: 将当前目录文件移到上级目录的 backup 文件夹
./	当前目录的路径前缀 (明确指定“当前目录下”)	./start.sh: 执行当前目录下的可执行脚本 (避免与系统命令冲突) - gcc -o app ./main.c: 编译当前目录下的 main.c 文件 - cd ./subdir: 切换到当前目录的 subdir 子目录 (等同于 cd subdir, 更清晰)
~	当前登录用户的家目录 (用户专属默认目录)	- cd ~: 快速回到家目录 (如 /home/ubuntu 或 /root) - ls ~/Downloads: 查看家目录下的 Downloads 文件夹 - touch ~/notes.txt: 在家目录创建 notes.txt 文件
/	系统根目录 (所有目录的最顶层目录)	- cd /: 切换到根目录 - ls /etc: 查看根目录下的 etc 系统配置目录 - cat /var/log/syslog: 查看根目录下的系统日志文件
-	上一次工作目录 (切换目录前的原目录)	- cd -: 在“当前目录”和“上一次目录”之间快速切换 - cp file.txt -: 将文件复制到上一次所在的目录



ise.....

Directory Commands

(2) (在上页)

mkdir -p a/b/c
创建多级

mkdir -m 755 xxx
755 权限

* mkdir directory-list

* Example: mkdir personal manage
创建了2个目录

* rmdir directory-list

* Directory must be empty

* Example: rmdir personal manage

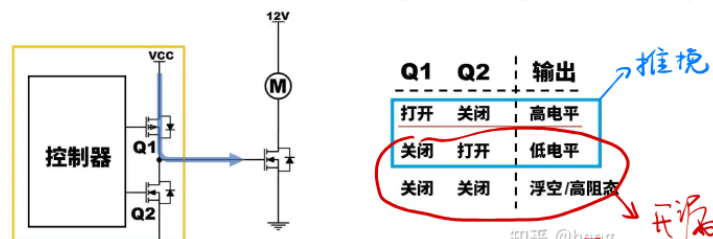


图212 数据时钟时序图

I²C: 0通信开始: SCL保持高平, SDA从高到低
 CPHA=1
 start condition 表示通信开始

M发出7位地址 + { 0 主从从入, 1 主从从出 }

地址后是应答位(ACK), 从设备根据其地址进行识别并向主~
 发送应答信号

Δ I²C 每次传8位(1字节), 接收石会发一个应答位

写: 主→从
 读: 从→主

7位 I²C 0x00-0x07 禁用 2⁷-1=127
 0x78-0x7F

查看
 改
 改
 Li

